

Architecture

```
+++      ++      +-+
| Operator  |      | llm_planner/ |      | schema/          |
| NL prompt | ->| planner.py   | ->| validator.py      |
|           |      | (qwen2.5:7b) |      | validate_mission |
+++      ++      +-+

one call, no          |
retries, no          | REJECTED --> stop,
execution power      | print errors, exit 1
                    |
                    v  VALID mission dict

++
| formation/          |
| formation_planner.py |
| -> {agent_id: [(x,y)...]} |
+---+--+
      |
      v

++
| schema/validator.py  |
| validate_formation_  |
| separation / _geofence |
+---+--+
REJECTED --> stop      | VALID
(never simulates)      v
++
| executor/executor.py |
| build_target_trajectories |
| (deterministic, no LLM) |
+---+--+
      |
      v

++
| sim/sim_runner.py    |
| gym-pybullet-drones  |
| CtrlAviary + DSLPIDControl|
+---+--+
      |
      v

logs/*.csv telemetry +
min-separation report
```

`run.py` drives all of this and prints each stage's output as it happens see `README.md` for a sample run.

Why the Boundaries Are Where They Are

The LLM only ever runs once, and only ever produces JSON. `llm_planner/planner.py` is the *only* file in this codebase that talks to an LLM (a local Ollama server over plain HTTP see `docs/design_decisions.md` §14 for why Ollama instead of the Anthropic API this was originally written against). It has one function, `plan_mission()`, that takes a string and returns a string. It does not know about the formation layer, the executor, or the simulator, and it is never called again after the mission JSON exists. This is what makes “the LLM must never issue commands directly”

true structurally, not just by convention there is no code path from the model call back into vehicle control.

Validation is a hard gate, not a warning. `schema/validator.py`'s functions return a `ValidationResult` / raise `MissionValidationError`; they never silently coerce a bad plan into something runnable. `run.py` checks `.valid` after both the schema/safety stage and, later, the post-formation separation/geofence stage, and stops the pipeline on either failure. A rejected mission produces a clear list of *why* it was rejected and nothing else happens no partial execution, no “best effort” fallback.

The formation layer is pure geometry, not simulation. `formation/formation_planner.plan_formation()` takes a validated mission dict and returns `{agent_id: [(x, y), ...]}` plain 2D points, no altitude, no speed, no PyBullet types anywhere in the return value. That means it can be (and is) unit tested with nothing but Python floats see `tests/test_formation_geometry.py` and `tests/test_formation_split` without importing `gym-pybullet-drones` at all.

The executor is the only place mission JSON meets the simulator, and even there it doesn't reach into `sim/` internals `executor/executor.py` calls `sim.sim_runner.run_simulation()` with a fully-formed set of trajectories and initial positions; `sim_runner.py` doesn't know where those came from. Same mission JSON in, same formation output, same trajectories, same simulation, every time (see `docs/design_decisions.md` §10 for exactly what guarantees that).

Data Contract at Each Boundary

From	To	Shape
Operator	llm_planner.plan_mission	str (natural language)
llm_planner.plan_mission	schema.validate_mission	str (raw JSON text)
schema.validate_mission	formation.plan_formation	dict matching mission_schema_already_vali-dated
formation.plan_formation	schema.validate_formation_separation / _geofence	dict[str, list[tuple float]]
formation.plan_formation	executor.build_target_trajectories	same shape, once stage-2 val-idation passes
executor.build_target_trajectories	sim.run_simulation	dict[str, np.ndarray] (per-agent (N, 3) XYZ target trajectories) + initial

Every arrow above is a plain-data boundary (strings, dicts, tuples, small dataclasses) nothing upstream of the executor imports `gym_pybullet_drones`, and nothing downstream of `llm_planner` talks to an LLM at all.